Dispositivos ubicuos en una red distribuida

IPFS + Web3-Arduino + sensores/actuadores + seguridad física

Del P2P y DHT al mundo físico

La misma pregunta cambia de escala

BitTorrent

¿Cómo distribuimos bloques entre muchos peers?

DHT

¿Cómo encontramos información sin un índice central?

IPFS

¿Cómo identificamos y verificamos contenido por lo que es?

Ahora agregamos el mundo físico

¿Cómo hacemos que sensores y actuadores sean peers verificables de una red distribuida?

Modelo mental: una red cyber-física distribuida



Principio

El dispositivo no es una “cosa conectada”: es un participante con estado, identidad, capacidades y efectos físicos.

1 · El dispositivo está expuesto

Modelo de amenaza: el atacante puede tocar el hardware

- Manipular entradas de sensores o señales físicas.
- Intentar extraer claves criptográficas de memoria.
- Modificar o reemplazar el firmware.
- Atacar interfaces de depuración, alimentación o comunicaciones.
- Intentar provocar fallos para saltarse verificaciones.
- Coludirse con operadores locales o usuarios legítimos.

Regla de diseño

No asumimos que el extremo físico es confiable.

Desconfianza mutua

Ni el dispositivo ni el operador son automáticamente confiables

Dispositivo

Puede estar comprometido, clonado o manipulado.

Operador

Puede tener acceso físico y motivos para sabotear.

Red

Puede perder mensajes, aislar peers o mentir sobre disponibilidad.

Objetivo

Reducir la confianza necesaria: verificar identidad, integridad, autorización y evidencia.

Emparejamiento reemparejamiento seguro: Resurrecting Duckling

Identidad inicial y reemplazo controlado

① Primer contacto

El dispositivo aprende una identidad autorizada.

② Operación

Acepta solamente al peer emparejado.

③ Falla física

El componente debe ser reemplazado.

④ Resurrección

Procedimiento autorizado + nueva identidad + auditoría.

Pregunta de seguridad

¿Quién tiene autoridad para cambiar la identidad física que el dispositivo acepta?



Identidad: del dispositivo físico al peer lógico

Hardware

serial / eFuse / secure element

Clave

private key + public key

Identidad

ID derivado / dirección / PeerID

Política

qué puede hacer

Idea clave

Identidad ≠ ubicación. Una dirección de red puede cambiar; una identidad criptográfica puede permanecer.

Identidad en microcontroladores: SRAM PUF

Physical Unclonable Function basada en el estado inicial de la memoria RAM

1. SRAM PUF (Firma del Silicio)

Las diferencias físicas microscópicas de fabricación hacen que cada celda de SRAM tenga una tendencia reproducible a arrancar en un valor (0 o 1) al encenderse.

```
Arranque 1: 101101001110...
Arranque 2: 101101001100... (inestable)
Arranque 3: 101101001110...
```

Flujo de Generación de Claves:
Propiedades físicas → SRAM PUF → Helper Data + ECC → Clave criptográfica

2. Identificador Único vs. Identidad Criptográfica

A. Identificador Único (Público): Sirve únicamente para indexar y nombrar al dispositivo (ej. "Dispositivo #12345"). No es un secreto y puede ser copiado libremente.

B. Identidad Criptográfica (Secreta): Permite probar matemáticamente la autenticidad ("Soy realmente el dispositivo #12345 y lo demuestro mediante este desafío-respuesta"). Requiere un secreto.

★ *La gran ventaja de la PUF es que la clave secreta no se almacena de forma permanente en memoria no volátil, se reconstruye bajo demanda y desaparece al apagar el chip.*

Mecanismo	¿Es Único?	¿Es Secreto?	Uso Principal
UID / Chip ID / MAC Address	**SÍ**	**NO**	Identificación básica de red e inventario
OTP / eFuse	**SÍ**	**A veces**	Inyección de claves e identidad de fábrica
Clave en Secure Element	**SÍ**	**SÍ**	Almacenamiento seguro y autenticación fuerte
PUF SRAM / Dedicada	**SÍ**	**Deriva secreto**	**Derivación de clave secreta sin almacenamiento físico**
Key Gen en Chip (HW)	**SÍ**	**SÍ**	Criptografía asimétrica y autenticación basada en hardware

Idea clave: La SRAM PUF aprovecha imperfecciones físicas inevitables para crear identidades seguras sin necesidad de inyectar claves de forma externa.

Tamper resistance ≠ tamper evidence

Dos objetivos diferentes

Tamper resistance

Dificultar o impedir que el atacante alcance secretos.

Ej.: encapsulado, hardware seguro, mallas activas.

Tamper evidence

Hacer que una manipulación deje una evidencia detectable.

Ej.: sellos, trazas, eventos de auditoría.

En una red distribuida

La evidencia física puede convertirse en un evento verificable por otros participantes.

Tres estrategias físicas

Elegir según amenaza, costo y mantenimiento

Potting

Resina epoxi u otro encapsulado.

Dificulta acceso físico y puede hacer la manipulación destructiva.

Malla activa

Patrones conductores alrededor del material sensible.

Una interrupción puede disparar respuesta de seguridad.

Sellado

Evidencia física de apertura o sustitución.

Útil cuando la inspección y auditoría son parte del modelo.

Trade-off

Más protección física suele significar más costo, menor reparabilidad y un ciclo de mantenimiento diferente.

Canales laterales: el hardware habla sin querer

El secreto puede filtrarse por física

Tiempo

Variaciones en duración de operaciones.

Potencia

SPA / DPA: consumo eléctrico.

EM / óptico

Radiación electromagnética o señales ópticas.

Acústico / térmico

Vibraciones, sonido y temperatura.

Pregunta

¿Puede el atacante aprender una clave sin leer directamente la memoria?

Fault injection: no necesito leer la clave

Puedo intentar hacer que el procesador se equivoque

Glitch

Transitorio de reloj o alimentación.

Fault

Una operación produce un resultado inesperado.

Bypass

Intento saltar una comparación o una política.

Impacto

Código no autorizado.

Defensa

Verificación redundante + hardware de seguridad + detección de estado anómalo.

Mitigación de side-channels

Diseño criptográfico + diseño físico

- Blinding: aleatorizar la entrada para romper correlaciones entre secreto y observable físico.
- Masking: transformar internamente los valores para que una sola observación no revele el secreto.
- Implementaciones constantes: reducir dependencias secretas del tiempo de ejecución.
- Filtrado y desacoplo: reducir emisiones y variaciones observables.
- Blindaje físico: limitar observación electromagnética y acceso a señales.

Mitigación de side-channels

Diseño criptográfico + diseño físico

Blinding

Aleatorizar la entrada para romper correlaciones entre secreto y observable físico.

Masking

Transformar internamente los valores para que una sola observación no revele el secreto.

Constantes

Reducir dependencias secretas del tiempo de ejecución en las implementaciones.

Filtrado

Reducir emisiones y variaciones observables mediante técnicas de desacoplo.

Blindaje

Limitar la observación electromagnética y el acceso físico a las señales.

Enfoque Integrado

La seguridad efectiva contra ataques de canal lateral requiere combinar defensas a nivel de algoritmo (criptografía) con barreras físicas y de hardware.

Secure Boot: ¿qué firmware tiene derecho a ejecutarse?

ROM / raíz
Código inicial confiable.

Bootloader
Verifica siguiente etapa.

Firmware
Imagen firmada.

Aplicación
Solo arranca si la cadena es válida.

ESP32
Secure Boot usa material protegido en eFuse y verifica la cadena de arranque; Secure Boot y Flash Encryption son funciones separadas.

Secure Boot + Flash Encryption

Integridad y confidencialidad son problemas distintos

Secure Boot

Protege la integridad de lo que se ejecuta.

Pregunta: “¿Esta imagen está autorizada?”

Flash Encryption

Protege la confidencialidad de lo almacenado en flash.

Pregunta: “¿Puede el atacante leer los secretos?”

Combinación

Para un dispositivo expuesto, ambas propiedades pueden ser necesarias.

Raíz de confianza y atestación

Demostrar más que “tengo una clave”

Identidad

¿Qué dispositivo es?

Integridad

¿Qué firmware ejecuta?

Estado

¿Está en una configuración aceptable?

Prueba

Evidencia verificable.

Resultado

Una red distribuida puede aplicar políticas sobre dispositivos, no solo sobre direcciones IP.

El ciclo de vida: el firmware también es un objeto distribuido

Build
Firmware + manifest

Sign
Firma del fabricante

Distribute
Red / gateway / IPFS

Verify
Dispositivo valida antes de instalar

Idea
IPFS puede distribuir el artefacto; la firma y la política determinan si el dispositivo puede instalarlo.

Hardware Seguro: Los 4 mejores MCU del mercado

Análisis comparativo de características de seguridad, aceleración criptográfica y aislamiento

STM32U5

PROCESAMIENTO Y CRIPTO

Cortex-M33 @ 160 MHz
TRNG (NIST)
AES-256 (side-channel), HASH, PKA, OTFDEC

AISLAMIENTO Y ALMACENAMIENTO

TrustZone completa + MPU
Excel. (HUK + TF-M ITS)
Muy listo (STSAFE)

ECOSISTEMA Y SOFTWARE

TF-M, STM32Trust, SBSFU, PSA/SESIP L3

Por qué destaca:

Mejor equilibrio general, bajo consumo y el ecosistema de desarrollo más maduro.

LPC55S6x / MCX N

PROCESAMIENTO Y CRIPTO

Cortex-M33 (hasta dual)
TRNG disponible
AES-256, SHA-2, CASPER (RSA/ECC), PRINCE

AISLAMIENTO Y ALMACENAMIENTO

TrustZone + MPU
Muy fuerte (PUF + EdgeLock)
Integrado EdgeLock / SE ext.

ECOSISTEMA Y SOFTWARE

MCUXpresso, TF-M, EdgeLock 2GO

Por qué destaca:

Sobresaliente aceleración criptográfica y robusto enclave seguro por hardware.

nRF54L Series

PROCESAMIENTO Y CRIPTO

Cortex-M33
TRNG (NIST)
CRACEN (AES-256, SHA-3, ECC, RSA, ChaCha20)

AISLAMIENTO Y ALMACENAMIENTO

TrustZone + MPU
Fuerte (KMU ampliado)
Listo para integración externa

ECOSISTEMA Y SOFTWARE

nRF Connect SDK, TF-M, MCUboot/NSIB

Por qué destaca:

La opción moderna ideal para dispositivos inalámbricos con alta seguridad.

PIC32CK SG

PROCESAMIENTO Y CRIPTO

Cortex-M33 @ 120 MHz
TRNG disponible
HSM completo (AES, SHA, RSA/ECC 4096)

AISLAMIENTO Y ALMACENAMIENTO

TrustZone + MPU
Excelente (HSM dedicado)
HSM integrado (enclave propio)

ECOSISTEMA Y SOFTWARE

MPLAB Harmony, secure boot/update

Por qué destaca:

Máxima integración de seguridad gracias a su módulo HSM por hardware dedicado.

¿Por qué un ESP32 no es un nodo blockchain completo?

- Memoria, almacenamiento y energía son recursos limitados.
- Mantener una cadena completa y ejecutar el consenso de una red pública no es el objetivo del dispositivo.
- El dispositivo puede actuar como cliente ligero o consumidor de APIs/RPC.
- La criptografía local sí puede ser viable: firmas, hashes y operaciones de clave.
- El diseño debe separar “participar en una aplicación blockchain” de “ejecutar toda la infraestructura blockchain”.

No confundir

Blockchain ≠ magia de seguridad para sensores.

web3-arduino: un puente entre ESP32 y Ethereum

ESP32 + Arduino

Firmware C/C++
Wi-Fi
sensores / actuadores

web3-arduino

JSON-RPC
calls a contratos
firmado de transacciones

Nodo Ethereum

RPC / cliente accesible

Contrato

estado + reglas

Estado del proyecto

https://github.com/AlphaWallet/Web3E

¿Qué puede hacer web3-arduino?

La interfaz es pequeña; la arquitectura detrás no

- Consultar información del cliente Ethereum mediante JSON-RPC.
- Calcular hashes y preparar datos de contratos.
- Consultar funciones de contratos (“call”).
- Construir y firmar transacciones localmente.
- Enviar la transacción a un nodo remoto.
- Trabajar con una clave privada asociada a una dirección.

Pregunta de seguridad

¿Dónde vive la clave privada y qué ocurre si alguien obtiene acceso físico al ESP32?

Arquitectura realista: sensor → decisión → registro → acción

Sensor
lectura física

ESP32
firma + política

Red
P2P / HTTP / RPC

Registro
chain / IPFS

Actuador
efecto físico

Dos planos
Plano de datos: mediciones y artefactos. Plano de control: identidades, permisos, políticas y comandos.

Blockchain + IPFS: no guardar todo on-chain

Datos grandes

logs · imágenes · series temporales · firmware

IPFS

CID → contenido direccionado
verificación de integridad

Blockchain

hash/CID · permisos · eventos · estado

Patrón

on-chain: evidencia y reglas · off-chain: contenido y volumen

IPFS como capa de datos de una red de sensores

Sensor A
medición → objeto

Sensor B
medición → objeto

Sensor C
medición → objeto

IPFS
CID + DAG + peers proveedores

Ventaja
El contenido puede ser recuperado desde distintos peers; el CID permite verificar qué objeto se recibió.

Una red distribuida de sensores necesita tres verdades

Identidad · contenido · mundo físico

1. Identidad

“Este mensaje fue firmado por el dispositivo X”.

2. Contenido

“Estos bytes corresponden al CID Y”.

3. Realidad física

“La medición representa realmente el fenómeno Z”.

La tercera no se resuelve solamente con criptografía

Necesitamos calibración, redundancia, attestation, procedimientos y modelos de confianza.

Actuadores: el problema se vuelve físico

Leer datos es distinto de provocar una acción

Sensor

“temperatura = 80°C”

Política

“si > 75°C → enfriar”

Actuador

encender bomba / abrir válvula

Control distribuido

La autorización del comando debe depender de identidad, contexto, estado y política.

Smart contracts como política, no como reflejo del mundo

Blockchain

estado compartido
permisos
transiciones

Gateway / peer

interpreta política
coordina
cachea

ESP32

interlocks
límites físicos
acción local

Regla de seguridad

Nunca asumir que una instrucción remota sustituye a los límites de seguridad locales.

Amenazas de extremo a extremo

Físico
extracción de claves / manipulación

Firmware
bootloader o aplicación alterados

Identidad
clonación / suplantación

Red
MITM / aislamiento / pérdida

Datos
inyección / replay / falsificación

Control
comando no autorizado

Defensa en profundidad
Ninguna capa resuelve por sí sola todo el problema.

Principios de diseño

Una checklist para dispositivos ubicuos

- Identidad única y claves protegidas.
- Pairing inicial y reemplazo controlado.
- Secure Boot + protección de almacenamiento cuando el riesgo lo exige.
- Actualizaciones firmadas, verificadas y con anti-rollback.
- Mensajes autenticados y protección contra replay.
- Separar datos, política y control físico.
- Usar IPFS para contenido direccionado; blockchain para estados/evidencia apropiados.
- Asumir que sensores pueden fallar o mentir: calibración + redundancia.
- Mantener límites de seguridad locales aunque la red falle.

Preguntas para la discusión

Para cerrar la transición de DHT/IPFS hacia sistemas cyber-físicos

- Si un sensor firma un dato falso, ¿qué sabemos realmente?
- ¿Qué aporta IPFS que no aporta una blockchain?
- ¿Dónde debería vivir una clave privada de un actuador?
- ¿Qué pasa si el dispositivo está offline durante horas?
- ¿Qué parte de una política debe estar on-chain y cuál local?
- ¿Cómo reemplazamos un sensor sin permitir que cualquiera se haga pasar por él?
- ¿Qué evidencia necesitamos para afirmar que una medición es confiable?

Fuentes y lecturas recomendadas

Fuentes usadas para completar los gaps

- Texto base proporcionado: seguridad física, side channels, secure boot y microcontroladores.
- NISTIR 8259A — IoT Device Cybersecurity Capability Core Baseline.
- NIST IR 8259 Rev. 1 (2026) — Foundational Cybersecurity Activities for IoT Product Manufacturers.
- IETF RFC 9019 — A Firmware Update Architecture for Internet of Things.
- Espressif ESP-IDF — Secure Boot v2 / Flash Encryption / eFuse.
- kopanitsa/web3-arduino — Arduino/ESP32 Web3 library for Ethereum (repositorio original, actualmente marcado como unmaintained).
- AlphaWallet/Web3E — evolución/rewrite para dispositivos embebidos Arduino/ESP32.
- IETF RFC 8613 — OSCORE para protección end-to-end en dispositivos restringidos.

NIST: https://csrc.nist.gov/pubs/ir/8259/a/final
NIST 2026: https://csrc.nist.gov/pubs/ir/8259/r1/final
RFC 9019: https://www.rfc-editor.org/info/rfc9019/
Espressif Secure Boot:
https://docs.espressif.com/projects/esp-idf/en/latest/esp32/security/secure-boot-v2.html
web3-arduino: https://github.com/kopanitsa/web3-arduino
Web3E: https://github.com/AlphaWallet/Web3E
OSCORE: https://www.rfc-editor.org/rfc/rfc8613.html